

SPM-Kit · Fathom

Usage Guide

Version 0.1.4 (Alpha) — in-tree 0.1.5.dev0

2026-07-27

Released wheel: 0.1.4 (DOI 10.5281/zenodo.21303280)

Author: José Labarca Baeza

License: MIT

Repository: <https://github.com/kegouro/spmkit>

Site: <https://kegouro.github.io/spmkit>

Canonical documentation: This PDF is a printable companion. The Markdown version at `docs/user-guide.md` is the authoritative reference and may contain additional detail. Both are verified against SPM-Kit 0.1.4 (development tree 0.1.5.dev0).

Acknowledgements: Tomás Corrales and the SPM Lab at UTFSM provided selected experimental datasets and laboratory context during development and evaluation. María Saavedra Fredes and Benjamin Schleyer helped locate and share candidate datasets for the validation campaign. Developed without dedicated institutional funding.

SPM-Kit is an open-source Python toolkit for analysing AFM/KPFM scanning-probe-microscopy data. Fathom is its desktop workspace. This guide covers installation, conceptual model, supported formats, the Fathom GUI, CLI reference, Python API, scientific validation, and extensibility.

This open-source project has been developed without dedicated institutional funding. Development status: Alpha (3 — Alpha). API may change before 1.0.

Contents

1	Introduction	3
1.1	What is SPM-Kit?	3
1.2	What is Fathom?	3
1.3	Component relationship	3
1.4	What SPM-Kit does <i>not</i> do	3
1.5	Status	3
2	Installation	3
2.1	Core only	4
2.2	GUI / Fathom	4
2.3	All features	4
2.4	Extras matrix	4
2.5	Development installation	4
2.6	Headless / HPC	4
2.7	Platform notes	4
3	Verifying the installation	5
4	Conceptual model	5
4.1	Architecture	5
4.2	Data model	5
4.3	Force-spectroscopy domain	5
4.4	Result objects	5
5	Supported file formats	5
5.1	Maturity vocabulary	5
5.2	What “validated” means for .nid	6
6	Quick start	6
7	Fathom desktop workspace	6
7.1	Launching	6
7.2	Opening data	6
7.3	Projects (.spmproj)	6
7.4	Appearance	7
7.5	Command palette	7
8	Perspectives	7
8.1	Image perspectives	7
8.2	Force perspectives	7
8.3	Other perspectives	8
9	End-to-end workflows	8
9.1	Topography leveling and roughness	8
9.2	Force-curve contact mechanics	8
9.3	Force-volume property mapping	8
9.4	Thermal tune	9
9.5	Evaporation / mass sensing	9
9.6	KPFM	9
9.7	Other workflows	9

10 Command-line reference	9
10.1 Common options	10
11 Python API	10
11.1 Public imports	10
11.2 Loading and inspecting	10
11.3 Analysis	10
11.4 Force spectroscopy	10
11.5 Export and figures	10
11.6 Recipes	11
12 Keyboard shortcuts	11
13 Exports and provenance	11
13.1 Export formats	11
13.2 Provenance	11
14 Scientific validation	12
14.1 Evidence-level vocabulary	12
14.2 Status summary	12
14.3 Gwyddion as reference	13
14.4 Simulator	13
15 Safety and interpretation warnings	13
16 Extensibility	14
16.1 Entry-point groups	14
16.2 Adding a format	14
16.3 Adding a perspective	14
17 Troubleshooting	14
18 Development and quality	14
19 Relationship to SPM-Kit Data Hunter	15
20 Citation, license, and contributions	15
20.1 How to cite	15
20.2 License	15
20.3 Contributing	15
20.4 Funding	15
20.5 Project status	15
A Quick reference card	15
B CLI command index	16
C Perspective index	16
D Supported formats matrix	17
E Keyboard shortcuts	17

1 Introduction

1.1 What is SPM-Kit?

SPM-Kit is an open-source Python toolkit for analysing AFM/KPFM scanning-probe-microscopy data. It reads raw instrument files and produces publication-ready results: roughness (ISO 25178), KPFM contact-potential statistics, nanomechanical property maps, thermal-tune cantilever calibration, grain detection, spectral/fractal analysis, and single-molecule force spectroscopy.

SPM-Kit was developed independently by José Labarca Baeza at Universidad Técnica Federico Santa María (UTFSM), informed by practical work with AFM/SPM data, and is released under the MIT license.

1.2 What is Fathom?

Fathom is the desktop workspace application bundled with SPM-Kit. It is a PyQt6 + pyqtgraph GUI organised into *perspectives* (task-oriented views), not flat tabs. A command palette (**Ctrl+K**) provides keyboard access to every function.

1.3 Component relationship

```
spmkit (Python package)
  spmkit.core    -- numerical engine (pure Python, no UI imports)
  spmkit.cli     -- command-line interface (Typer + Rich)
  spmkit.gui     -- Fathom desktop workspace (PyQt6 + pyqtgraph)
```

- **spmkit** is the Python package and CLI entry point.
- **Fathom** is the graphical workspace.
- **SPM-Kit** is the project/ecosystem name.

1.4 What SPM-Kit does *not* do

- It is **not a microscope controller**.
- It is **not a certified metrology tool**.
- It does **not** provide medical, clinical, or regulatory reports.
- Simulation features are **educational**, not quantitative digital twins.

1.5 Status

SPM-Kit is **alpha-stage** (Development Status 3). The `.nid` image parsers are cross-checked against Gwyddion at machine precision — this is *parser parity* with another implementation of the same formula, not physical-model validation. The roughness parameters `Sa`, `Sq` and `Sz` are independently **cross-validated** against Gwyddion 2.71 (**LEVEL 3**, 18/18 comparisons over six synthetic surfaces); KPFM, spectral and the remaining image analyses are exercised by unit and synthetic-recovery tests (**LEVEL 1/2**). Force-spectroscopy workflows are tested on synthetic and lab data but have *not* been independently cross-validated. Nanoscope III `.spm` image reading is numerically verified against Gwyddion 2.71 at matrix precision (**LEVEL 2**, partial). Some readers (`afmformats`, JPK TIFF, Bruker `.00N`) remain experimental (see §14).

2 Installation

Requirements: **Python** `>= 3.11`.

2.1 Core only

```
pip install spmkit
```

Installs: numpy, typer, rich, pyyaml. Sufficient for CLI, Python API, and all image analysis.

2.2 GUI / Fathom

```
pip install "spmkit[gui]"
```

Adds: PyQt6, pyqtgraph, matplotlib, Crameri colormaps, matplotlib-scalebar.

2.3 All features

```
pip install "spmkit[all]"
```

2.4 Extras matrix

Extra	Provides	Deps
gui	Fathom desktop workspace	PyQt6, pyqtgraph, matplotlib
viz	Publication figures, colormaps	matplotlib, cmcrameri, scalebar
gwy	Gwyddion .gwy read/write	gwyfile
hdf5	HDF5 import/export	h5py
grains	Grain/particle detection	scipy
report	HTML/PDF reports	Jinja2 + spmkit[viz]
nanosurf	Validated .nhf reader	NSFOpen
afm	JKP QI, .ibw, HDF5, NT-MDT	afmformats
jpgk	JKP TIFF force curves	tiffle
parallel	Parallel force-volume	joblib
pandas	DataFrame export	pandas
dev	Lint, test, type-check	pytest, ruff, mypy, black
test-gui	GUI test runner	pytest-qt
docs	Documentation build	mkdocs-material
all	All of the above	—

2.5 Development installation

```
git clone https://github.com/kegouro/spmkit
cd spmkit
pip install -e ".[all,dev]"
```

2.6 Headless / HPC

Install core only: `pip install spmkit`. Add [viz] for headless figure generation.

2.7 Platform notes

Developed and tested on **macOS** and **Linux**. Windows is not regularly tested. Fathom requires a display; set `QT_QPA_PLATFORM=offscreen` for headless tests.

3 Verifying the installation

```
spmkit --version          # spmkit 0.1.4 (in-tree install reports 0.1.5.dev0)
python -c "import spmkit; print(spmkit.__version__)"
```

4 Conceptual model

4.1 Architecture

Three strictly separated layers (enforced by `tests/test_architecture.py`):

- `core/` — pure Python, zero UI imports.
- `cli/` — Typer + Rich. Only imports the public `core` API.
- `gui/` — PyQt6 + pyqtgraph. Only imports the public `core` API.

4.2 Data model

```
from spmkit import load, SPMDData, SPMChannel

data: SPMDData = load("scan.nid")
ch: SPMChannel = data["Z-Axis"]

ch.data          # 2D ndarray in physical units
ch.unit          # "m", "V", "{\textdegree}"
ch.x_range       # metres
ch.y_range       # metres
ch.direction     # "forward" / "backward"
```

4.3 Force-spectroscopy domain

For force curves: `ForceCurve`, `ForceSegment`, `ForceVolume` (lazy-loaded), `Calibration`.

4.4 Result objects

Every analysis returns an immutable dataclass with a `.to_dict()` method, serialisable via `spmkit.core.export`.

5 Supported file formats

Extension	Source	Type	Access	Status
.nid	NanoSurf	Images + force-volume	Read	Validated
.nhf	NanoSurf HDF5	Images	Read	Tested
.gwy	Gwyddion	Images	Read+Write	Tested
.nid (spec)	NanoSurf	Force-volume	Read	Tested
.jpk-force	JPB/Bruker	Single curve	Read	Tested
.jpk (TIFF)	JPB/Bruker	Force-map/QI	Read	Experimental
.001/.002	Bruker SPM	Images	Read	Experimental
.spm	Bruker/Nanoscope III	Images	Read	Partial (LEVEL 2)
.ibw	Asylum	Force curves	Read	Experimental

5.1 Maturity vocabulary

Validated Numerical output cross-checked against independent reference.

Tested Unit and synthetic-recovery tests pass.

Experimental Reader exists but lacks thorough testing.

5.2 What “validated” means for .nid

The physical conversion $phys = \text{Dim2Min} + (raw + 2^{31})/2^{32} \cdot \text{Dim2Range}$ has been cross-checked against Gwyddion exports at **correlation 1.000000** for topography, phase, and sensor channels. This is *parser/conversion parity* with another implementation of the same formula — it does not, by itself, validate a physical model. The independently cross-validated quantities at defined tolerance are the ISO 25178 roughness parameters Sa, Sq and Sz (LEVEL 3, vs Gwyddion 2.71, 18/18 over six synthetic surfaces). See docs/TRACEABILITY.md and the external validation register.

6 Quick start

```
# Inspect a file
spmkit info scan.nid

# Roughness (ISO 25178)
spmkit roughness scan.nid -c Z-Axis --level plane

# Publication figure
spmkit figure scan.nid -c Z-Axis -o topo.png --colormap batlow

# Launch Fathom
spmkit gui scan.nid

# Force curve fit
spmkit forcecurve curva.jpk-force --model dmt --tip-radius 2e-8
```

Python:

```
from spmkit import load
from spmkit.core.analysis import leveling, roughness

data = load("scan.nid")
flat = leveling.plane_fit(data["Z-Axis"])
stats = roughness.statistics(flat)
print(f"Sa = {stats.sa:.3g} {stats.unit}")
```

7 Fathom desktop workspace

7.1 Launching

```
spmkit gui          # Fathom (default)
spmkit gui scan.nid # open file on launch
spmkit gui --legacy # legacy 7-tab app (fallback)
```

7.2 Opening data

Drag and drop a file onto the window, or **Ctrl+O**. Fathom inspects the file and routes image channels to image perspectives and force/spectroscopy data to force perspectives. If a file contains both, Fathom asks which to open.

7.3 Projects (.spmproj)

Ctrl+S saves the open file, pipeline parameters, and active perspective as a human-editable YAML file.

7.4 Appearance

- **Ctrl+Shift+L** — toggle light/dark theme.
- **Ctrl+Shift+A** — appearance dialog (theme presets, accent colour, font size).
- Settings persist between sessions.
- The theme feeds the Qt app, pyqtgraph, and matplotlib simultaneously.

7.5 Command palette

Ctrl+K, then type a command name and press Enter. Indexes all perspectives and registered commands.

8 Perspectives

Fathom presents 12 perspectives through 6 modules. Each perspective shows only the panels relevant to its workflow.

8.1 Image perspectives

Module: image	
image	Channel viewer, leveling (plane/poly/align-rows), colormap, line profile ROI, roughness + KPFM analysis, profile export.
grains	Particle/grain detection on levelled topography; colour overlay, size statistics. Requires grains extra.
spectral	Radial PSD (log-log), fractal dimension D, Hurst exponent H, correlation length.
resonance	Thermal tune: SHO fit $\rightarrow f_0$, Q, spring constant by equipartition.
evaporation	Mass sensing: load series of tuning .nid files $\rightarrow f(t)$, mass, d^2 law fit.

8.2 Force perspectives

Module: force	
force	Navigate curves, fit Hertz/DMT/Sneddon, inspect modulus $\pm 1\sigma$, adhesion, dissipation, R^2 . Editable pipeline, Monte Carlo uncertainty. Export scientific CSV.
smfs	Single-molecule force spectroscopy: prominence-based rupture detection, WLC/FJC polymer fits, contour-length histogram.
map	Force-volume $\rightarrow$ property maps (modulus, adhesion, contact, R^2) + histogram. Vectorised (CPU/GPU) or per-curve pipeline.
batch	Process folders of force curves $\rightarrow$ summary CSV.

8.3 Other perspectives

Module: figure	
figure	WYSIWYG publication figure editor: title, axes, colormap, scale bar, draggable annotations. Export PNG/SVG/PDF.
Module: view3d	
view3d	3D topography surface with hillshade lighting, visual Z exaggeration.
Module: simulator	
simulator	Educational cantilever digital twin: thermal noise spectrum, resonance shift with added mass.

9 End-to-end workflows

9.1 Topography leveling and roughness

CLI:

```
spmkit roughness scan.nid -c Z-Axis --level plane
```

Python:

```
from spmkit import load
from spmkit.core.analysis import leveling, roughness

data = load("scan.nid")
flat = leveling.plane_fit(data["Z-Axis"])
stats = roughness.statistics(flat)
print(stats.sa, stats.sq, stats.sz, stats.unit)
```

GUI: Drag file → Image perspective → select channel → Plane Fit → read results from Analysis panel.

Caveats

Roughness depends on scan size, pixel density, and leveling method. Always report these alongside roughness values.

9.2 Force-curve contact mechanics

```
spmkit forcecurve sample.jpg-force --model dmt --tip-radius 2e-8
```

Contact models: sphere (Hertz), paraboloid (Sneddon paraboloid), cone (Sneddon cone), dmt (Hertz + adhesion offset). The experimental JKR model (`spmkit jkr`) fits adhesive contact but has not been validated against an independent reference.

Caveats

Modulus depends on tip radius (uncertain), contact model (approximation), and calibration. Report all three alongside the result. JKR is experimental.

9.3 Force-volume property mapping

```
spmkit forcemap volume.nid --model dmt --fast -f maps.png
spmkit forceexport volume.nid -o ./results
```

Creates property maps (modulus, adhesion, contact point, R^2), histograms, and CSV exports. Use `-pipeline` for variable-length curves (QI data), `-parallel` for multi-core, `-backend gpu` for GPU (requires CuPy).

9.4 Thermal tune

Load a thermal spectrum in the Thermal Tune perspective or use:

```
from spmkit.core.analysis.mechanics import thermal_spring_constant
```

9.5 Evaporation / mass sensing

```
spmkit evaporation ./tuning/ -k 0.3 -o evap.csv
```

Tracks $f(t) \rightarrow \text{mass} \rightarrow \text{evaporation rate}$. Fits the d^2 law for diffusion-limited evaporation.

9.6 KPFM

```
spmkit analyze scan.nid --tip-wf 4.8 -o ./results
```

Reports mean CPD, standard deviation, and sample work function ($\Phi_{\text{sample}} = \Phi_{\text{tip}} - e \cdot \text{CPD}$).

9.7 Other workflows

```
spmkit grains scan.nid --min-size 10           # grain detection
spmkit psd scan.nid -c Z-Axis                  # fractal analysis
spmkit batch ./measurements/ -o summary.csv    # batch processing
spmkit fbatch ./curves/ -o batch.csv           # force batch
spmkit figure scan.nid -o topo.png --colormap batlow # figure export
```

10 Command-line reference

19 commands are registered in the Typer application.

Command	Description
<code>info</code>	Show file metadata and channels
<code>roughness</code>	ISO 25178 parameters (Sa, Sq, Sz, Ssk, Sku)
<code>psd</code>	Spectral: fractal dimension, Hurst, correlation length
<code>analyze</code>	Full pipeline: roughness + KPFM $\rightarrow$ CSV + JSON
<code>nanomech</code>	Force-curve fit (Hertz/DMT/Sneddon) $\rightarrow$ Young's modulus
<code>grains</code>	Grain/particle detection and size statistics
<code>batch</code>	Process folder $\rightarrow$ summary CSV
<code>evaporation</code>	Mass sensing: $f(t) \rightarrow \text{mass}$, evaporation rate, d^2 law
<code>figure</code>	Publication figure: PNG/SVG/PDF with scale bar
<code>convert</code>	Format conversion (<code>.nid</code> $\rightarrow$ <code>.gwy</code> / <code>.h5</code>)
<code>verify</code>	<code>.nid</code> byte-level integrity audit (8 checks)
<code>gui</code>	Launch Fathom (or legacy with <code>-legacy</code>)
<code>workspace</code>	Force-curve workspace (redesign)
<code>forcecurve</code>	Fit single force curve (JPK/NanoSurf)

<code>forcemap</code>	Force-volume → property maps
<code>forcereport</code>	Force-volume report (HTML/LaTeX/PDF)
<code>forceexport</code>	Full export bundle to folder
<code>fbatch</code>	Batch force processing → CSV
<code>jkr</code>	[Experimental] JKR adhesive contact fit

10.1 Common options

- `--channel / -c` — channel name (default: Z-Axis for images, Deflection for force)
- `--output / -o` — output path
- `--level / -l` — leveling: `plane`, `poly`, `none`
- `--model` — contact model: `sphere`, `paraboloid`, `cone`, `dmt`
- `--tip-radius` — tip radius in metres (default: `1e-08`)

Run `spmkit <command> -help` for full options.

11 Python API

11.1 Public imports

```
from spmkit import load, SPMData, SPMChannel, __version__
from spmkit.core.analysis import leveling, roughness, kpfm, mechanics
from spmkit.core.analysis import spectral, grains, resonance
from spmkit.core.export import to_csv, to_json, to_hdf5
from spmkit.core.viz import FigureSpec, save_figure
```

11.2 Loading and inspecting

```
data = load("scan.nid")      # auto-detect format
print(data.names)           # channel names
ch = data["Z-Axis"]         # SPMChannel
```

11.3 Analysis

```
flat = leveling.plane_fit(data["Z-Axis"])
stats = roughness.statistics(flat)
cpd = kpfm.statistics(data["CPD"], tip_work_function=4.8)
frac = spectral.fractal_dimension(flat)
grains_result = grains.detect(flat, min_size=4)
```

11.4 Force spectroscopy

```
from spmkit.core.io.forceload import load_nid_force
from spmkit.core.analysis.mechanics import fit_hertz

volume = load_nid_force("sample.nid")
result = fit_hertz(volume.curves[0], tip_radius=10e-9, model="dmt")
print(result.young_modulus, result.r_squared)
```

11.5 Export and figures

```
to_csv(stats, "roughness.csv")
to_json(stats, "roughness.json")
to_hdf5(data, "scan.h5")
```

```
spec = FigureSpec(title="AFM Topography", colormap="batlow")
save_figure(flat, spec, "topo.png")
```

11.6 Recipes

```
from spmkit.core.pipeline import Recipe, Step, run

recipe = Recipe(steps=(
    Step(op="calibrate"),
    Step(op="find_contact_point"),
    Step(op="fit_elasticity", params={"model": "dmt", "tip_radius": 2e-8}),
))
_, ctx = run(recipe, curve)
print(ctx["young_modulus"], ctx["r_squared"])
```

12 Keyboard shortcuts

Shortcut	Action
Ctrl+K	Command palette
Ctrl+O	Open file
Ctrl+S	Save project
Ctrl+M	Calculate map
Ctrl+Shift+R	Generate report
Ctrl+Shift+L	Toggle light/dark theme
Ctrl+Shift+A	Customize appearance
Ctrl+Left / Ctrl+Right	Previous / next curve
Ctrl+1 ... Ctrl+5	Tab switch (legacy app only)

13 Exports and provenance

13.1 Export formats

Format	Via	Notes
CSV	to_csv(), CLI	Units in headers; empty cells for NaN
JSON	to_json(), CLI	Structured; programmatic consumption
HDF5	to_hdf5(), convert	Self-describing binary; requires hdf5 extra
.gwy	spmkit convert	Round-trip with Gwyddion
PNG/SVG/PDF	spmkit figure	Publication-quality figures
HTML/PDF	spmkit forcereport	Full force-volume report
YAML	core.pipeline	Reproducible pipeline recipe

13.2 Provenance

- `SPMData.source_path` records the original file path.
- `SPMData.metadata` preserves instrument parameters.
- `spmkit verify` performs 8 byte-level integrity checks on .nid files.
- YAML recipes capture the exact pipeline parameters.
- Exports include metadata when the format supports it.

14 Scientific validation

SPM-Kit uses a layered approach and reports evidence with an explicit level vocabulary. “Validated” is not used as a blanket label: a capability is described by the strongest evidence that actually applies to it.

1. **Unit tests** — function correctness on synthetic input.
2. **Synthetic recovery** — known-property data → verify recovery.
3. **Cross-checks** — compare against Gwyddion for the same input (parser/conversion parity).
4. **Instrument-file loading** — real lab files load without errors.
5. **Numerical verification (LEVEL 2)** — matrix-precision agreement with an independent reference on a declared set of files (e.g. Nanoscope III .spm, 6 files vs Gwyddion 2.71).
6. **Cross-validation (LEVEL 3)** — externally registered campaign comparing derived quantities at defined tolerance (currently Sa, Sq, Sz vs Gwyddion 2.71; 18/18 over six synthetic surfaces).
7. **Experimental** — exists but lacks thorough testing or independent cross-check.

14.1 Evidence-level vocabulary

LEVEL 1 (Unit tested) Function-level tests pass on synthetic input.

LEVEL 2 (Numerically verified) Matrix-precision agreement with an independent reference on a declared file set.

LEVEL 3 (Cross-validated) Externally registered campaign comparing derived quantities at a defined tolerance, with a frozen script and frozen data set.

Experimental Reader or analysis exists without independent cross-check.

14.2 Status summary

Capability	Evidence	Status	Limitations
.nid image parsing	vs Gwyddion: corr 1.000000	Cross-checked	Parser parity; signed 32-bit LE; other dtypes untested
.nid physical conv.	8 integrity checks	Cross-checked	Parser/conversion parity; assumes Dim2Range linear
.nid orientation	flipud for Y channels	Cross-checked	Parser parity; non-image channels as-is
Roughness Sa/Sq/Sz (synth.)	vs Gwyddion 2.71, 18/18, 6 surfaces	Cross-validated (L3)	Synthetic surfaces, plane-levelled
Roughness ISO 25178 (general)	Synthetic σ recovery	Tested	Ssk/Sku and general surfaces: synthetic-recovery only
Nanoscope III .spm	vs Gwyddion 2.71, 6 files	Numerically verified (L2)	Partial; scaling unresolved returns raw
KPFM CPD statistics	Unit tests	Tested	Work function: tip calibration
Hertz/DMT force fit	Synthetic recovery	Tested	Tip radius uncertainty dominant
Force-volume maps	Unit + synthetic tests	Tested	Grid regularity assumed

Thermal tune	Unit tests	Tested	Equipartition assumption
Evaporation	Unit tests	Tested	Spherical droplet; diffusion-limited
Grain detection	Unit tests	Tested	Threshold-based; no shape metrics
Fractal/PSD	Unit tests	Tested	Self-affine model; finite bandwidth
JKR adhesive contact	No external ref	Experimental	Use with caution
.nhf reader	Unit tests	Tested	Generic HDF5 walker
.gwy reader/writer	Round-trip test	Tested	Verified
.jpk-force reader	Unit tests	Tested	Calibration from metadata
JPK QI (.jpk TIFF)	Limited testing	Experimental	Requires jpk extra
Bruker SPM (.001)	Limited testing	Experimental	Requires afm extra
afmformats readers	Limited testing	Experimental	Quality varies by format
SMFS polymer fits	Unit tests	Tested	Model choice critical
Fathom GUI	Headless GUI tests (tests/gui)	Tested	Qt version/platform dependent
Architecture	AST test	Enforced	No runtime enforcement

14.3 Gwyddion as reference

Gwyddion serves as an *independent reference implementation*, not universal ground truth. Both tools implement their own conversion from the same byte stream. The cross-validated campaign (LEVEL 3, Sa/Sq/Sz) is externally registered with a frozen script and frozen data set at <https://github.com/kegouro/spmkit-validation>; it does not constitute physical-model validation.

14.4 Simulator

The cantilever simulator is **educational only**. It models a damped harmonic oscillator with additive noise. Not suitable for calibration or metrology.

15 Safety and interpretation warnings

- **Results require domain judgment.** High R^2 does not guarantee correct model choice.
- **Calibration values matter.** Young’s modulus depends on tip radius (uncertain), spring constant (calibration), and contact model (approximation).
- **Check units.** `SPMChannel.unit` tells the physical unit.
- **Experimental readers may misinterpret metadata.** Format parsing is best-effort for reverse-engineered formats.
- **Fitted parameters are not automatically physical truth.**
- **SPM-Kit is not a medical, clinical, regulatory, or safety-critical instrument.**

- **Preserve original data.** SPM-Kit reads without modifying files.

16 Extensibility

16.1 Entry-point groups

Group	Purpose
<code>spmkit.plugins.v1</code>	Readers, analyses, domains
<code>spmkit.gui.modules</code>	Fathom modules (perspectives + panels)

16.2 Adding a format

Implement the Reader Protocol (`inspect` + `load`) and register via `spmkit.plugins.v1`.

16.3 Adding a perspective

Define a `ModuleSpec` with `PanelSpec` and `PerspectiveSpec`, register via `spmkit.gui.modules`. See `docs/extending.md` for full details.

17 Troubleshooting

Missing GUI extra

```
pip install "spmkit[gui]"
```

Qt platform plugin error

Install Qt dependencies or set `QT_QPA_PLATFORM=offscreen`.

Unsupported format

Check extension matches `.nid/.nhf/.gwy/.jpk-force`. For experimental readers, install `afm` or `jpk`.

Missing optional dependency

Install the relevant extra (e.g. `pip install "spmkit[grains]"`).

Large force-volume memory

Use lazy loading or the `-fast` CPU vectorised path.

GPU fallback

CuPy is not bundled. Install separately: `pip install cupy-cuda12x`.

Export failure

Check write permissions. Use `-output` with a writable path.

18 Development and quality

```
git clone https://github.com/kegouro/spmkit
cd spmkit
pip install -e ".[all,dev,test-gui]"
pre-commit install
```

Quality checks (CI gate):

```
make check      # lint + types + tests
make lint       # ruff check
make type       # mypy (strict on core)
make test       # pytest with coverage
```

GUI tests (headless):

```
QT_QPA_PLATFORM=offscreen pytest tests/gui -q
```

Documentation:

```
pip install -e ".[docs]"
python -m mkdocs build --strict
```

19 Relationship to SPM-Kit Data Hunter

SPM-Kit Data Hunter is a companion project that discovers and curates public data candidates for validation. It does **not** analyse data and does **not** validate SPM-Kit output.

- **SPM-Kit / Fathom** — analyse data.
- **Data Hunter** — discover data for validation.
- **Discovery is not validation.**

20 Citation, license, and contributions

20.1 How to cite

```
@software{spmkit2026,
  author      = {Jos\'e Labarca Baeza},
  title       = {spmkit: Open-source AFM/KPFM analyser
                 for scanning probe microscopy},
  year        = {2026},
  version     = {0.1.4},
  doi         = {10.5281/zenodo.21303280},
  url         = {https://github.com/kegouro/spmkit},
}
```

20.2 License

MIT. See LICENSE in the repository root.

20.3 Contributing

Issues and pull requests at <https://github.com/kegouro/spmkit/issues>. See CONTRIBUTING.md.

20.4 Funding

This open-source project has been developed without dedicated institutional funding.

20.5 Project status

Alpha (released 0.1.4; in-tree 0.1.5.dev0). Core functionality is stable and tested. APIs may change before 1.0. New contributors and format readers are welcome.

A Quick reference card

```
# Install
pip install "spmkit[gui]"

# Inspect
spmkit info scan.nid

# Analyse
spmkit roughness scan.nid -c Z-Axis --level plane
spmkit psd scan.nid -c Z-Axis
spmkit analyze scan.nid -o ./results --tip-wf 4.8
```

```

# Figures
spmkit figure scan.nid -o topo.png --colormap batlow

# Convert
spmkit convert scan.nid scan.gwy

# Verify
spmkit verify scan.nid

# Force
spmkit forcecurve curve.jpk-force --model dmt --tip-radius 2e-8
spmkit forcemap volume.nid --fast -f maps.png
spmkit forcereport volume.nid -o report
spmkit forceexport volume.nid -o ./export
spmkit fbatch ./curves/ -o batch.csv

# Evaporation
spmkit evaporation ./tuning/ -k 0.3 -o evap.csv

# Grains
spmkit grains scan.nid --min-size 10

# GUI
spmkit gui [file]

```

B CLI command index

Command	Category	Input
info	Inspection	.nid, .nhf
roughness	Image	.nid, .nhf, .gwy
psd	Spectral	.nid, .nhf, .gwy
analyze	Pipeline	.nid, .nhf, .gwy
nanomech	Force	.nid (spectroscopy)
grains	Image	.nid, .nhf, .gwy
batch	Batch	Folder of SPM
evaporation	Resonance	Folder of .nid tuning
figure	Figure	.nid, .nhf, .gwy
convert	Format	.nid
verify	Audit	.nid
gui	GUI	— (optional file)
workspace	GUI	Force-curve file
forcecurve	Force	.jpk-force, .nid
forcemap	Force	.nid (force-volume)
forcereport	Force	.nid (force-volume)
forceexport	Force	.nid (force-volume)
fbatch	Batch	Folder of force curves
jkr	Force (exp.)	Calibrated curve

C Perspective index

Key	Label	Module	Type
image	Imagen	image	Image
grains	Granos	image	Image
spectral	Espectral	image	Image
resonance	Sintoni'a te'rmica	image	Image
evaporation	Evaporacio'n	image	Image
force	Curva de fuerza	force	Force
smfs	SMFS	force	Force
map	Mapa	force	Force
batch	Batch	force	Force
figure	Figura	figure	Figure
view3d	Vista 3D	view3d	Image
simulator	Simulador	simulator	Educational

D Supported formats matrix

Extension	Source	Type	Access	Status
.nid	NanoSurf	Images + FV	Read	Validated
.nhf	NanoSurf HDF5	Images	Read	Tested
.gwy	Gwyddion	Images	Read+Write	Tested
.nid (spec)	NanoSurf	Force-volume	Read	Tested
.jpk-force	JPK/Bruker	Single curve	Read	Tested
.jpk (TIFF)	JPK/Bruker	Force-map/QI	Read	Experimental
.001/.002	Bruker SPM	Images	Read	Experimental
.spm	Bruker/Nanoscope III	Images	Read	Partial (LEVEL 2)
.ibw	Asylum	Force curves	Read	Experimental

E Keyboard shortcuts

Shortcut	Action
Ctrl+K	Command palette
Ctrl+O	Open file
Ctrl+S	Save project
Ctrl+M	Calculate map
Ctrl+Shift+R	Generate report
Ctrl+Shift+L	Toggle theme
Ctrl+Shift+A	Appearance settings
Ctrl+Left / Ctrl+Right	Previous / next curve
Ctrl+1 ... Ctrl+5	Tab switch (legacy app)